

基于轻量化深度学习的浙江河道水质智能监测与预警系统详细设计报告

版本号	修改日期	修改作者	修改内容	组号
V2.0	2026 年 4 月 30 日	单智屹	初始版本创建	13

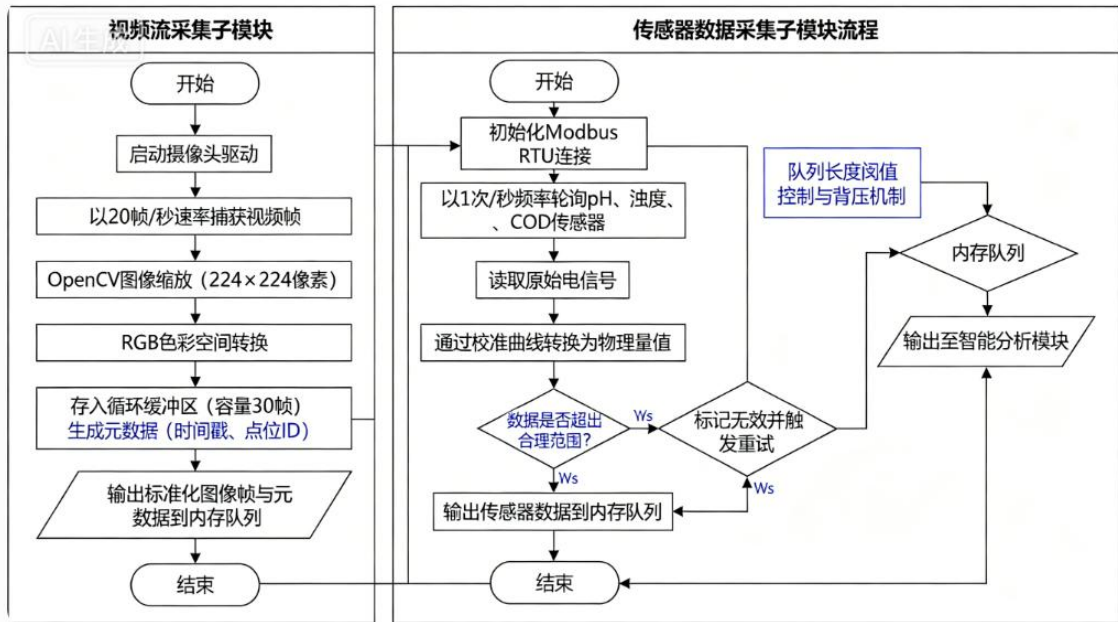
一、功能模块设计

1.1 数据采集模块详细设计

数据采集模块负责从河道监测点的摄像头与各类水质传感器获取原始数据，是整个系统的数据入口。该模块运行于边缘端设备，包括视频流采集子模块与传感器数据采集子模块。

视频流采集子模块采用 OpenCV 库调用摄像头驱动，以每秒 20 帧的速率实时捕获视频帧。该子模块内部维护一个固定大小的循环缓冲区，容量为 30 帧，确保内存使用可控。每捕获一帧后，进行图像缩放与归一化处理，将原始图像统一调整为 224×224 像素大小，并转换为 RGB 色彩空间，以满足轻量化模型的输入要求。处理后的图像帧存入缓冲区，同时生成包含时间戳与点位 ID 的元数据。

传感器数据采集子模块通过 Modbus RTU 协议连接 pH 传感器、浊度传感器与 COD 传感器。该子模块以每秒一次的频率轮询各传感器，读取原始电信号后通过内置校准曲线转换为物理量值。读取到的传感器数值经过异常值过滤处理，如超出预设合理范围的数据将被标记为无效并触发重试机制。有效数据与对应时间戳打包为 JSON 格式，传递给智能分析模块。



数据采集模块与智能分析模块之间的调用关系为异步生产者-消费者模式。

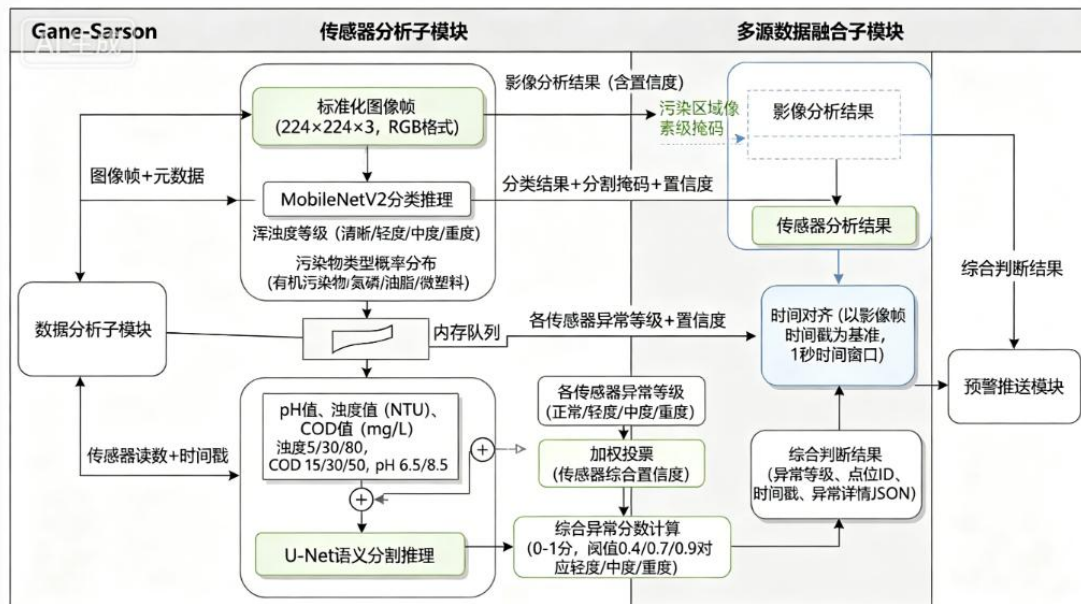
数据采集模块作为生产者，将标准化后的图像帧与传感器数据放入内存队列；智能分析模块作为消费者，从队列中取出数据进行处理。当队列长度超过阈值时，数据采集模块会暂时降低采集频率，防止内存溢出。本模块的数据流方向为从硬件设备到边缘端内存队列，不涉及数据库操作，确保采集延迟最小化。

1.2 智能分析模块详细设计

智能分析模块是系统的核心计算单元，负责对采集到的多源数据进行融合分析并输出水质判定结果。该模块划分为影像分析子模块、传感器分析子模块与多源数据融合子模块。

影像分析子模块加载预训练的 MobileNetV2 与 U-Net 组合模型。MobileNetV2 负责图像分类任务，输入 224×224 像素图像后，经深度可分离卷积层处理，输出浑浊度等级，包括清晰、轻度浑浊、中度浑浊、重度浑浊四种等级，以及污染物类型概率分布，涵盖有机污染物、氮磷污染、油脂污染、微塑料四种类型。U-Net 负责语义分割任务，输出图像中污染区域的像素级掩码，用于定位污染位置。两个模型并行执行，总推理时间控制在 150 毫秒以内。推理完成后，该子模块输出包含分类结果、分割掩码与置信度的数据结构。

传感器分析子模块接收 pH 值、浊度值与 COD 值，与预设阈值进行比较。浊度值超过 5 NTU 判定为轻度异常，超过 30 NTU 判定为中度异常，超过 80 NTU 判定为重度异常。COD 值超过 15 mg/L 为轻度异常，超过 30 mg/L 为中度异常，超过 50 mg/L 为重度异常。pH 值低于 6.5 或高于 8.5 触发异常标记，异常程度根据偏离幅度分级。该子模块输出各传感器的异常等级与综合置信度。



多源数据融合子模块采用时间对齐后加权投票的策略。具体而言，该子模块以 1 秒为时间窗口，收集该窗口内的影像分析结果与传感器分析结果。影像分析结果的权重设置为 0.6，传感器分析结果的权重设置为 0.4，这是由于影像信息能够提供更直观的水质状态判断。加权投票后，系统计算综合异常分数，分数在 0 到 0.4 之间判定为正常，0.4 到 0.7 之间判定为轻度异常，0.7 到 0.9 之间判定为中度异常，0.9 以上判定为重度异常。融合完成后，该模块将综合判断结果传递给预警推送模块。智能分析模块与预警推送模块之间的调用方式为函数调用，传递参数为异常等级、点位 ID、时间戳与异常详情 JSON。

1.3 预警推送模块详细设计

预警推送模块负责根据智能分析模块输出的异常等级生成告警内容，并通过多渠道完成通知推送。该模块划分为告警生成子模块、消息分发子模块与推送状态追踪子模块。

告警生成子模块接收异常等级、点位 ID、时间戳与异常详情后，首先查询监测点配置信息，获取点位名称、地理位置与负责人联系方式。然后根据异常等级选择对应的告警模板，轻度异常模板采用蓝色标记，内容为简要提示；中度异常模板采用黄色标记，内容包含异常类型与建议措施；重度异常模板采用红色标记，内容包含详细异常数据与紧急处置指引。告警内容生成后，系统为每条告警分配唯一的告警 ID，并写入 MySQL 数据库。

图需要提供时间序列与对应数值，该子模块从数据记录中提取时间戳与指定水质指标，生成 x 轴为时间、y 轴为指标值的系列数据。热力图展示需要聚合各监测点的平均异常分数，该子模块根据点位经纬度坐标与异常分数生成散点数据，设置颜色映射规则，分数越高颜色越偏红。

地图展示子模块调用高德地图 API 或 Leaflet 开源库，在页面上渲染监测点位分布。该子模块负责将监测点坐标转换为地图坐标系，并生成自定义标记点。每个标记点支持点击交互，点击后弹出信息窗口，显示该点位的最新水质状态与历史数据链接。本模块的输出通过 RESTful API 返回给前端 Vue 应用，前端负责实际调用 ECharts 进行图表渲染。

1.5 权限管理模块详细设计

权限管理模块接收用户凭证，校验角色与权限后返回访问控制结果，其他模块在关键操作前均需调用本模块进行鉴权。该模块划分为身份认证子模块、授权校验子模块与用户管理子模块。

身份认证子模块采用 JWT 令牌机制实现无状态认证。用户登录时提交用户名与密码，系统验证凭证正确性后，生成 JWT 令牌，令牌中包含用户 ID、用户名、角色列表与过期时间，令牌有效期为 24 小时。后续请求中，客户端需在 HTTP 请求头的 Authorization 字段携带该令牌。本模块对每个请求进行令牌签名验证与有效期检查，验证失败时返回 401 状态码。

授权校验子模块基于角色的访问控制模型实现。系统预定义三种角色，包括管理员、普通用户与只读用户。管理员拥有全部操作权限，包括监测点管理、用户管理与系统配置。普通用户拥有其所辖监测点的数据查看与告警确认权限。只读用户仅拥有数据查看权限，无法进行任何修改操作。该子模块为每个 API 接口定义所需的权限级别，请求到达时提取 JWT 中的角色信息，与接口要求进行比对。用户管理子模块提供用户注册、信息修改、密码重置与角色分配功能。用户密码采用 BCrypt 算法进行哈希加密存储，明文密码不落盘。用户注册时需验证手机号与验证码，防止恶意注册。该子模块与前端交互遵循 RESTful 规范，所有敏感操作均需二次确认。

二、类设计

2.1 核心类结构设计

系统边缘端采用 Python 语言实现，云端后端采用 Java 语言实现。本节定义核心类的结构、属性、方法与继承关系。

边缘端定义 WaterQualityAnalyzer 类作为智能分析的主控类。该类包含 model_loader 属性，类型为 TF Lite 解释器实例；frame_buffer 属性，类型为双端队列，容量为 30；sensor_data 属性，类型为字典，存储最新传感器读数。该类提供 analyze_frame 方法，接收图像帧作为输入，返回分析结果字典；提供 analyze_sensor 方法，接收传感器读数列表，返回异常等级；提供 fuse_results 方法，接收影像与传感器分析结果，执行加权投票融合。

边缘端定义 ModelLoader 类负责 TensorFlow Lite 模型的加载与管理。该类继承自 Object 基类，包含 model_path 与 interpreter 两个属性。该类提供 load_model 方法，接收模型文件路径，初始化解释器并分配张量内存；提供 run_inference 方法，接收预处理后的图像数组，执行推理并返回输出张量；提供 quantize_input 方法，将浮点输入转换为模型要求的量化格式。ModelLoader 类采用单例模式设计，确保整个边缘端只存在一个模型加载器实例，避免重复加载造成内存浪费。

边缘端定义 DataCache 类负责网络中断时的本地数据缓存。该类包含 cache_dir 属性，指定本地缓存目录路径；max_cache_size 属性，限制最大缓存大小为 500 MB；cache_queue 属性，使用 Python 内置的 Queue 管理待上传数据。该类提供 save_to_cache 方法，将异常标签与元数据保存为 JSON 文件；提供 upload_cached_data 方法，网络恢复后遍历缓存目录并逐条上传；提供 clear_cache 方法，上传成功后删除已处理的缓存文件。

云端后端定义 User 实体类，对应数据库中的用户表。该类包含 id、username、password、phone、email、role、create_time、update_time 等私有属性，遵循 JavaBean 规范为每个属性提供 getter 与 setter 方法。该类还提供 isAdmin 方法，判断用户角色是否为管理员；isValid 方法，校验用户账号状态是否正常。User 类继承自 BaseEntity 基类，BaseEntity 包含公共的 id、createTime 与 updateTime 属性。

云端后端定义 MonitoringPoint 实体类，对应监测点配置表。该类包含 id、name、longitude、latitude、device_id、status、last_online_time 等属性。

该类提供 `isOnline` 方法，根据最后在线时间与当前时间差判断设备是否在线，超过 5 分钟未上报数据判定为离线。`MonitoringPoint` 类与 `User` 类之间不存在直接关联，但通过中间表 `user_point_permission` 建立多对多关系。

云端后端定义 `AlertRecord` 实体类，对应告警记录表。该类包含 `id`、`point_id`、`alert_level`、`alert_type`、`details`、`push_status`、`create_time`、`confirm_time`、`confirmed_by` 等属性。该类提供 `getAlertLevelDescription` 方法，将数值型异常等级转换为中文描述，轻度对应轻度异常，中度对应中度异常，重度对应重度异常。`AlertRecord` 类与 `User` 类之间存在多对一关系，一条告警记录可被多个用户确认，通过 `alert_confirmation` 表记录确认关系。

2.2 枚举与常量定义

系统定义 `AlertLevel` 枚举类型，包含 `NORMAL`、`MILD`、`MODERATE`、`SEVERE` 四个枚举常量。每个枚举常量包含 `levelCode` 与 `description` 两个字段，`levelCode` 为整数代码对应 0 到 3，`description` 为中文描述。枚举类提供 `fromCode` 静态方法，根据代码值返回对应的枚举常量。

系统定义 `SensorType` 枚举类型，包含 `PH`、`TURBIDITY`、`COD` 三个枚举常量。每个枚举常量包含 `unit` 字段，表示测量单位，`pH` 对应无单位，浊度对应 `NTU`，`COD` 对应 `mg/L`。枚举类提供 `getThreshold` 方法，返回每个传感器类型对应的异常阈值数组。

系统定义错误码常量类 `ErrorCode`，该类不可实例化，所有字段为静态常量。`SUCCESS` 定义为 0000，表示操作成功。`PARAM_ERROR` 定义为 1001，表示参数错误。`UNAUTHORIZED` 定义为 1002，表示未认证。`FORBIDDEN` 定义为 1003，表示无权限。`DATA_NOT_FOUND` 定义为 1004，表示数据不存在。`INFERENCE_FAILED` 定义为 2001，表示模型推理失败。`SENSOR_TIMEOUT` 定义为 2002，表示传感器读取超时。`MQTT_DISCONNECTED` 定义为 3001，表示 MQTT 连接断开。该类提供 `getMessage` 静态方法，根据错误码返回对应的提示信息。

2.3 集合数据结构设计

系统定义 `AnalysisResult` 数据结构，用于封装智能分析模块的输出结果。该结构体使用 Python 的 `dataclass` 装饰器定义，包含 `frame_id` 字段，类型为字符串；`timestamp` 字段，类型为 `datetime`；`turbidity_level` 字段，类型为整数，

范围为 0 到 3；pollution_types 字段，类型为列表，存储检测到的污染物类型字符串；confidence 字段，类型为浮点数，范围为 0 到 1；segmentation_mask 字段，类型为 numpy 数组，存储分割掩码。

系统定义 PushTask 数据结构，用于预警推送模块的任务队列。该结构体包含 task_id 字段，类型为字符串；alert_id 字段，类型为整数；channels 字段，类型为列表，存储需要推送的渠道名称；retry_count 字段，类型为整数，记录已重试次数；next_retry_time 字段，类型为 datetime，记录下次重试时间。该数据结构被放入优先级队列中，根据 next_retry_time 排序，优先处理最早需要重试的任务。

系统定义查询参数集合 QueryParams，用于封装前端传入的可视化查询条件。该类包含 point_ids 字段，类型为 List<Long>；start_time 与 end_time 字段，类型为 LocalDateTime；data_type 字段，类型为 String，可选值为 turbidity、cod、ph、alert_count；page 与 page_size 字段，类型为整数，用于分页控制。该类提供 validate 方法，校验时间范围不超过 30 天，分页参数不超过 100 条每页。

三、接口设计

3.1 模块间接口定义

数据采集模块向智能分析模块传递数据的接口定义为异步消息队列接口。输入参数为标准化后的图像帧，包含 frame_data 二进制数据、timestamp 时间戳、point_id 点位标识。输出参数为存入队列的成功状态，成功返回 true，队列满时返回 false 并触发背压控制。异常处理方面，当图像帧格式无法解析时，模块记录错误日志并丢弃该帧；当点位 ID 不存在时，模块拒绝接收数据。

智能分析模块向预警推送模块传递结果的接口定义为同步函数调用接口。接口方法签名为 pushAlert(AnalysisResult result)，输入参数为 AnalysisResult 对象，包含异常等级、点位 ID、时间戳与异常详情。输出参数为 AlertResult 对象，包含 alert_id 与 push_status。异常处理方面，当 AnalysisResult 对象为空时，接口直接返回空结果不触发推送；当异常等级为正常时，接口同样返回空结果不进行后续处理。推送失败时不抛出异常，而是将失败状态写入返回结果中。

接口精确伪代码定义如下：

【接口 1】智能分析模块 -> 预警推送模块

方法签名：AlertResult pushAlert(AnalysisResult result)

输入参数 AnalysisResult 结构定义：

```
{  
    pointId: string,          // 监测点 ID, 必填, 不能为空  
    timestamp: datetime,     // 时间戳, 必填  
    alertLevel: int,          // 异常等级, 取值 0-3, 0 表示正常  
    details: object,          // 异常详情 JSON, 包含浊度、COD、pH 等具体  
数值  
    confidence: float         // 置信度, 范围 0-1  
}
```

返回值 AlertResult 结构定义：

```
{  
    alertId: string,          // 生成的告警 ID, 失败时为空字符串  
    pushStatus: string        // 取值: SUCCESS, FAILED, SKIPPED  
}
```

异常处理：

- 当 result 为 null 时，抛出 IllegalArgumentException，错误码 1001
- 当 result.alertLevel == 0 时，直接返回 {alertId: "", pushStatus: "SKIPPED"}，不触发推送
- 数据库写入失败时，返回 {alertId: "", pushStatus: "FAILED"}，不抛异常

【接口 2】预警推送模块 -> 短信平台 API

请求方式：POST

URL: <https://api.smsprovider.com/v2/send>

请求头:

Content-Type: application/json

Authorization: Bearer {api_key}

请求体:

```
{
  "mobile": "13812345678",    // 手机号, 11 位数字
  "content": "【水质预警】XX 河道 COD 超标, 请及时处理", // 1-140 字符
  "sign": "水质监测"          // 短信签名
}
```

成功响应:

```
{
  "code": "SUCCESS",
  "messageId": "SG2026043012345678"
}
```

失败响应:

```
{
  "code": "BALANCE_NOT_ENOUGH",
  "message": "账户余额不足"
}
```

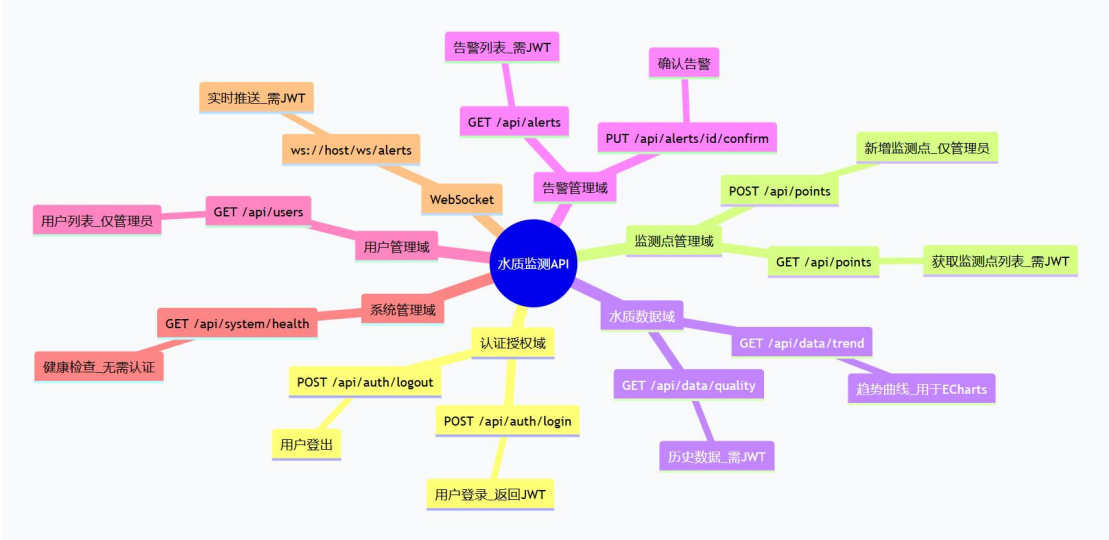
超时设置: 连接超时 3 秒, 读取超时 5 秒

重试策略: 失败后重试 3 次, 重试间隔为 1 秒、2 秒、4 秒 (指数退避)

智能分析模块向数据库写入数据的接口定义为数据持久化接口。接口方法签名为 `saveAnalysisRecord(AnalysisResult result)`, 输入参数为 `AnalysisResult` 对象。输出参数为布尔值, 表示写入是否成功。该接口采用异步写入方式, 将数据放入写入队列后立即返回, 不等待数据库确认。异常处理方面, 数据库连接中断时, 接口返回 `false` 并触发本地缓存机制。

预警推送模块调用短信平台 API 的外部接口定义为 HTTP POST 请求。请求 URL 为 `https://api.smsprovider.com/send`，请求头包含 Content-Type 为 `application/json` 与 Authorization 承载 API 密钥。请求体包含 `phone` 字段，字符串类型；`content` 字段，字符串类型，最大长度 140 字符；`signature` 字段，字符串类型，短信签名。返回值格式为 JSON，包含 `code` 状态码与 `message` 描述信息。异常处理方面，网络超时设置为 3 秒，超时后触发重试；返回码非成功时记录错误日志并进入重试队列。

后端服务向前端提供的 RESTful API 接口遵循统一规范。查询类接口采用 GET 方法，参数通过 URL 查询字符串传递。修改类接口采用 POST 或 PUT 方法，参数通过请求体以 JSON 格式传递。删除类接口采用 DELETE 方法。所有接口的响应格式统一为 `Result<T>` 结构，包含 `code` 整数状态码、`message` 字符串提示信息与 `data` 泛型数据。分页查询接口的响应额外包含 `total` 字段表示总记录数。接口鉴权方面，除登录接口外所有接口均需在请求头中携带 JWT 令牌。



3.2 接口异常处理与错误码

系统定义统一的错误码体系，所有接口在发生异常时返回对应的错误码。完整错误码定义如下表：

错误码	HTTP 状态码	错误类型	错误描述	前端 / 客户端处理建议
1001	400	客户端错误	请求参数缺失或格式错误	检查请求参数后重试

错误码	HTTP 状态码	错误类型	错误描述	前端 / 客户端处理建议
1002	401	客户端错误	JWT 令牌无效或已过期	跳转至登录页面
1003	403	客户端错误	用户权限不足	隐藏相关操作按钮,提示无权限
1004	404	客户端错误	请求的资源不存在	提示资源不存在
1005	405	客户端错误	请求方法不支持	检查 HTTP 请求方法类型
2001	500	服务端错误	模型推理引擎失败	记录日志,系统自动降级
2002	500	服务端错误	数据库连接异常	触发重试,失败后系统告警
2003	500	服务端错误	数据库查询超时	缩小查询范围后重新请求
2004	500	服务端错误	事务提交失败	记录日志,提示用户稍后重试
3001	503	设备端错误	摄像头离线	推送设备离线告警至运维端
3002	503	设备端错误	传感器读取超时	降级为纯影像分析模式
3003	503	设备端错误	边缘端存储空间不足	触发缓存淘汰机制或发起告警
4001	502	第三方错误	短信平台调用失败	进入重试队列,采用指数退避策略
4002	502	第三方错误	地图 API 调用超时	返回本地缓存数据或业务降级处理
9001	429	系统级错误	请求频率超过限制	延迟后重试,控制每秒最多 1 次请求

错误码	HTTP 状态码	错误类型	错误描述	前端 / 客户端处理建议
9002	503	系统级错误	内部服务调用超时	熔断当前服务, 返回兜底默认数据

客户端请求参数校验失败时, 接口返回 400 状态码, 错误码为 1001, 提示信息具体说明哪个参数不符合要求。用户未登录或 JWT 令牌过期时, 接口返回 401 状态码, 错误码为 1002, 前端收到此错误码后自动跳转至登录页面。用户已登录但权限不足时, 接口返回 403 状态码, 错误码为 1003, 前端收到后隐藏相关操作按钮。

3.3 通信协议规范

边缘端与服务端之间的通信采用 MQTT over TLS 协议。边缘端作为 MQTT 客户端, 服务端作为 MQTT Broker。数据传输主题定义为 `/wqi/{point_id}/{data_type}`, 其中 `point_id` 为监测点唯一标识, `data_type` 可选值为 `image_analysis` 或 `sensor_data`。消息载荷采用 `protobuf` 序列化格式, 相较于 `JSON` 具有更小的体积与更快的解析速度。TLS 配置方面, 边缘端需验证服务器证书, 禁用 `SSLv3` 与 `TLSv1.0` 等不安全协议。

服务端与前端之间的 `WebSocket` 通信用于实时告警推送。`WebSocket` 连接端点定义为 `/ws/alerts`, 客户端连接时需在子协议中携带 JWT 令牌用于身份认证。服务端在接收到推送需求时, 根据用户权限过滤后向对应连接发送消息。消息格式为 `JSON`, 包含 `type` 字段表示消息类型, `data` 字段承载具体内容。连接心跳间隔设置为 30 秒, 客户端需定时发送 `ping` 帧保持连接, 连续两次心跳超时则服务端主动断开连接。

服务端内部服务之间采用 `gRPC` 协议通信。`gRPC` 服务定义在 `.proto` 文件中, 使用 `protocol buffers` 定义消息格式与服务方法。智能分析服务与预警推送服务之间通过 `gRPC` 调用, 相比 `HTTP` 具有更低的延迟与更强的类型安全。`gRPC` 连接采用连接池管理, 每个服务实例与目标服务建立最多 10 条长连接, 减少连接建立开销。

四、数据库设计

4.1 核心表结构设计

用户表命名为 `sys_user`, 用于存储系统用户信息。该表包含 `id` 字段, 类型

为 BIGINT，设置为主键并自增；username 字段，类型为 VARCHAR(50)，设置为唯一索引，不允许为空；password 字段，类型为 VARCHAR(100)，存储 BCrypt 加密后的密码哈希值；phone 字段，类型为 VARCHAR(11)，设置为普通索引，用于登录与找回密码；email 字段，类型为 VARCHAR(100)，可为空；role 字段，类型为 VARCHAR(20)，取值为 admin、user 或 readonly；status 字段，类型为 TINYINT，0 表示禁用，1 表示启用；create_time 字段，类型为 DATETIME，默认当前时间；update_time 字段，类型为 DATETIME，自动更新。表引擎使用 InnoDB，字符集采用 utf8mb4 以支持完整 Unicode 字符。

监测点表命名为 monitoring_point，用于存储河道监测点配置信息。该表包含 id 字段，主键自增；name 字段，类型为 VARCHAR(100)，不允许为空；longitude 字段，类型为 DECIMAL(10,7)，存储经度坐标；latitude 字段，类型为 DECIMAL(10,7)，存储纬度坐标；device_id 字段，类型为 VARCHAR(50)，设置为唯一索引，对应边缘端设备序列号；status 字段，类型为 TINYINT，0 表示离线，1 表示在线；last_online_time 字段，类型为 DATETIME，记录最后一次收到数据的时间；location_desc 字段，类型为 VARCHAR(200)，存储地理位置描述信息。为加速空间查询，系统在 longitude 与 latitude 字段上建立复合索引。

水质数据表命名为 water_quality_data，用于存储每条分析记录。

```
CREATE TABLE water_quality_data (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '主键 ID',  
    point_id BIGINT NOT NULL COMMENT '监测点 ID, 关联 monitoring_point  
表',  
    timestamp DATETIME NOT NULL COMMENT '数据采集时间',  
    turbidity_level TINYINT NOT NULL DEFAULT 0 COMMENT '浑浊度等级,  
0-清晰, 1-轻度, 2-中度, 3-重度',  
    cod_value DECIMAL(10,2) NULL COMMENT 'COD 数值, 单位 mg/L, 传感器故障时可空',  
    ph_value DECIMAL(3,2) NULL COMMENT 'pH 数值, 传感器故障时可空',  
    pollution_types JSON NULL COMMENT '污染物类型 JSON, 格式:
```

```

["oil","microplastic"]',
    alert_level TINYINT NOT NULL DEFAULT 0 COMMENT '综合异常等级,
0-正常, 1-轻度, 2-中度, 3-重度',
    original_image_url VARCHAR(500) NULL COMMENT '原始图像对象存储
路径 (MinIO/COS)',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '记录创
建时间',

-- 外键约束
CONSTRAINT fk_water_quality_point FOREIGN KEY (point_id)
    REFERENCES monitoring_point(id) ON DELETE CASCADE ON UPDATE
CASCADE,

-- 索引定义
INDEX idx_point_time (point_id, timestamp),
INDEX idx_alert_level (alert_level),
INDEX idx_timestamp (timestamp)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='水质分析数据表'
PARTITION BY RANGE (YEAR(timestamp) * 100 + MONTH(timestamp)) (
    PARTITION p202601 VALUES LESS THAN (202602),
    PARTITION p202602 VALUES LESS THAN (202603),
    -- 每月新增分区, 保留最近 12 个月
);

```

告警记录表命名为 alert_record, 用于存储系统生成的告警信息。该表包含 id 字段, 主键自增; point_id 字段, 类型为 BIGINT, 设置为外键并建立索引; alert_level 字段, 类型为 TINYINT, 取值 1 到 3; alert_type 字段, 类型为 VARCHAR(50), 取值为 turbidity、cod、ph 或 combined; details 字段, 类型为 TEXT, 存储 JSON 格式的详细异常信息; push_status 字段, 类型为 VARCHAR(20), 取值为 pending、sent、failed 或 confirmed; create_time 字段, 类型为 DATETIME;

confirm_time 字段,类型为 DATETIME,可为空;confirmed_by 字段,类型为 BIGINT,关联 sys_user 表的 id。系统在 point_id 与 create_time 字段上建立复合索引,用于高效查询某个监测点的历史告警。

用户-监测点权限表命名为 user_point_permission,用于记录用户可查看的监测点范围。该表包含 id 字段,主键自增;user_id 字段,类型为 BIGINT,外键关联 sys_user;point_id 字段,类型为 BIGINT,外键关联 monitoring_point;create_time 字段,类型为 DATETIME。系统在 user_id 与 point_id 上建立复合唯一索引,确保同一用户对同一监测点只有一条权限记录。管理员角色不受此表限制,能够查看所有监测点数据。

4.2 缓存策略设计

系统采用 Redis 作为缓存层,缓存策略分为查询缓存与会话缓存两类。查询缓存针对可视化模块中的水质趋势曲线查询,将最近 5 分钟内的相同查询结果缓存起来。缓存键设计为 query:{point_ids_hash}:{start_time}:{end_time}:{data_type},值为序列化后的 ECharts 系列数据。缓存有效期设置为 5 分钟,数据更新时主动删除相关缓存键,保证缓存一致性。

会话缓存用于存储 JWT 令牌对应的用户信息。用户登录成功后,系统将用户 ID、用户名、角色列表等信息存入 Redis,键为 session:{user_id},值为用户信息 JSON 字符串,过期时间设置为 24 小时。后续请求中,每次需要用户信息时优先从 Redis 读取,避免频繁查询 MySQL 数据库。会话过期前 30 分钟,若用户仍有活跃请求,系统自动刷新过期时间。

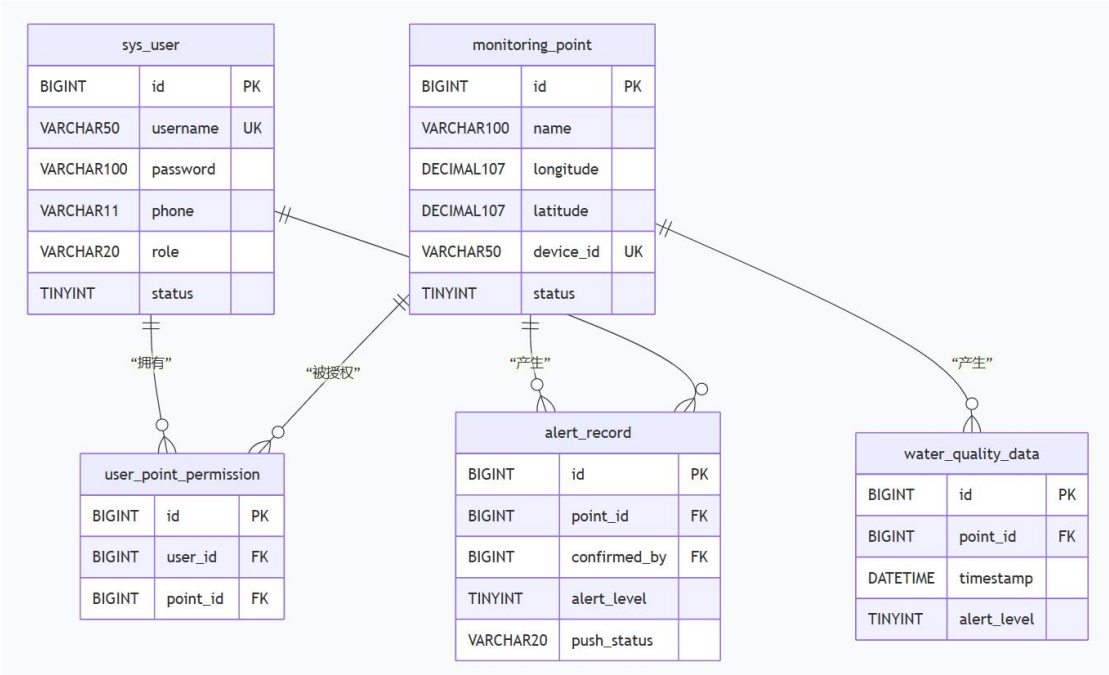
缓存更新采用旁路缓存模式。数据写入时,系统先更新 MySQL 数据库,再删除对应的缓存键。数据读取时,系统先查询 Redis,命中则直接返回;未命中则查询 MySQL,将查询结果写入 Redis 后再返回。对于告警记录等实时性要求较高的数据,系统设置较短的缓存有效期,甚至不使用缓存,确保用户看到最新的告警状态。

4.3 数据库性能优化

针对 water_quality_data 表数据量大的特点,系统采用按月分区的策略。分区键为 timestamp 字段,每个分区存储一个月的数据。当前月份的数据写入当

前分区，历史查询根据时间范围定位到具体分区，减少扫描数据量。同时，系统设置定期清理任务，保留最近 12 个月的数据，超过 12 个月的数据自动归档到冷存储。

索引设计遵循最左前缀原则。water_quality_data 表上建立 (point_id, timestamp) 复合索引，用于按点位查询时间序列数据。alert_record 表上建立 (point_id, create_time) 复合索引，用于按点位查询告警历史。所有索引均采用 BTREE 结构，避免使用哈希索引，因为系统大量使用范围查询而非等值查询。数据库连接池采用 HikariCP，配置最大连接数为 20，最小空闲连接数为 5，连接超时时间为 30 秒。对于读多写少的查询场景，系统配置主从复制，写操作在主库执行，读操作路由到从库，分散数据库负载。从库数量根据实际查询压力动态调整，初始配置为 1 个从库。



4.4 补充核心表

4.4.1 边缘端设备表 (edge_device)

```
CREATE TABLE edge_device (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    device_sn VARCHAR(64) NOT NULL UNIQUE COMMENT '设备序列号，出厂固  
化',  
    device_type VARCHAR(20) NOT NULL COMMENT '设备类型: raspberry_pi,
```

```

jetson_nano',

    firmware_version VARCHAR(20) NOT NULL DEFAULT '1.0.0' COMMENT '固件版本号',

    last_heartbeat DATETIME NULL COMMENT '最后心跳时间，用于判断在线状态',

    storage_usage_mb INT DEFAULT 0 COMMENT '边缘端存储已使用量(MB)',

    ip_address VARCHAR(45) NULL COMMENT '设备 IP 地址，支持 IPv6',

    point_id BIGINT NULL COMMENT '关联的监测点 ID，可空表示未绑定',

    status TINYINT DEFAULT 1 COMMENT '状态：0-离线，1-在线，2-故障',

    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_device_sn (device_sn),

    INDEX idx_point_id (point_id),

    INDEX idx_last_heartbeat (last_heartbeat),

    CONSTRAINT fk_edge_device_point FOREIGN KEY (point_id)

        REFERENCES monitoring_point(id) ON DELETE SET NULL

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='边缘端设备表';

```

4.4.2 系统配置表 (sys_config)

```

CREATE TABLE sys_config (

    id BIGINT PRIMARY KEY AUTO_INCREMENT,

    config_key VARCHAR(100) NOT NULL UNIQUE COMMENT '配置键，

    如:sms.api.url',

    config_value TEXT NOT NULL COMMENT '配置值，JSON 格式存储',

    description VARCHAR(500) NULL COMMENT '配置说明',

    is_sensitive TINYINT DEFAULT 0 COMMENT '是否敏感配置(密码/密钥)，1

    表示需加密存储',

    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
```

```
INDEX idx_config_key (config_key)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='系统配置表';
```

预置配置数据示例：

```
INSERT INTO sys_config (config_key, config_value, description) VALUES
('sms.api.url', 'https://api.smsprovider.com/v2/send', '短信平台 API
地址'),
('sms.retry.max', '3', '短信发送最大重试次数'),
('alert.cache.days', '7', '异常告警本地缓存保留天数');
```

五、业务流程细化

5.1 实时监测与预警流程

实时监测与预警流程是系统的核心业务场景。流程起始于边缘端设备初始化，摄像头驱动与传感器驱动加载完成后，系统进入主循环。每帧图像采集后立即送入推理队列，传感器数据则按固定间隔读取。推理队列中的任务由线程池中的工作线程并行处理，充分利用边缘端设备的计算资源。

模型推理完成后，系统将影像分析结果与最新的传感器读数进行时间对齐。对齐算法以影像帧的时间戳为基准，选取时间差在 500 毫秒以内的传感器读数进行融合，若无匹配的传感器读数则仅使用影像分析结果。融合后的综合异常分数与预设阈值比较，低于 0.4 则判定为正常，此时仅将数据写入本地日志，不触发后续预警流程。

多源数据融合算法详细伪代码：

```
def fuse_image_and_sensor(image_result, sensor_result,
time_window_ms=500):
    """
    image_result: 影像分析结果 {turbidity_level, pollution_probs,
```

```

timestamp}

    sensor_result: 传感器结果列表 [{turbidity, cod, ph,
timestamp}, ...]

    返回值: {final_score, confidence, alert_level}
    """

# 步骤1: 时间对齐, 寻找与影像时间戳最接近的传感器数据
matched_sensor = None
min_diff = time_window_ms

for sensor in sensor_result:
    diff = abs(sensor.timestamp -
image_result.timestamp).total_seconds() * 1000

    if diff < min_diff:
        min_diff = diff
        matched_sensor = sensor

# 步骤2: 计算传感器分数 (0-1)
if matched_sensor is None:
    sensor_score = 0
    sensor_weight = 0 # 无匹配的传感器数据时, 降级为纯影像
    confidence_penalty = 0.8
else:
    turb_score = min(matched_sensor.turbidity / 80.0, 1.0) # 80
NTU 对应满分
    cod_score = min(matched_sensor.cod / 50.0, 1.0) # 50
mg/L 对应满分
    ph_score = abs(matched_sensor.ph - 7.0) / 3.5 # pH
偏离7 越多分越高

# 传感器加权: 浊度 50%, COD30%, pH20%
sensor_score = turb_score * 0.5 + cod_score * 0.3 + ph_score *

```

0.2

```
    sensor_weight = 0.4
    confidence_penalty = 1.0

# 步骤 3: 计算影像分数
# 浑浊度等级转分数: 清晰 0, 轻度 0.33, 中度 0.67, 重度 1.0
turb_map = {0: 0.0, 1: 0.33, 2: 0.67, 3: 1.0}
turb_score = turb_map[image_result.turbidity_level]
max_pollution_prob = max(image_result.pollution_probs)
image_score = turb_score * 0.7 + max_pollution_prob * 0.3

# 步骤 4: 加权融合
final_score = image_score * (1 - sensor_weight) + sensor_score *
sensor_weight

# 步骤 5: 计算置信度
consistency = 1.0 - abs(image_score - sensor_score)
confidence = consistency * confidence_penalty

# 步骤 6: 确定异常等级
if final_score < 0.4:
    alert_level = 0 # 正常
elif final_score < 0.7:
    alert_level = 1 # 轻度异常
elif final_score < 0.9:
    alert_level = 2 # 中度异常
else:
    alert_level = 3 # 重度异常
```

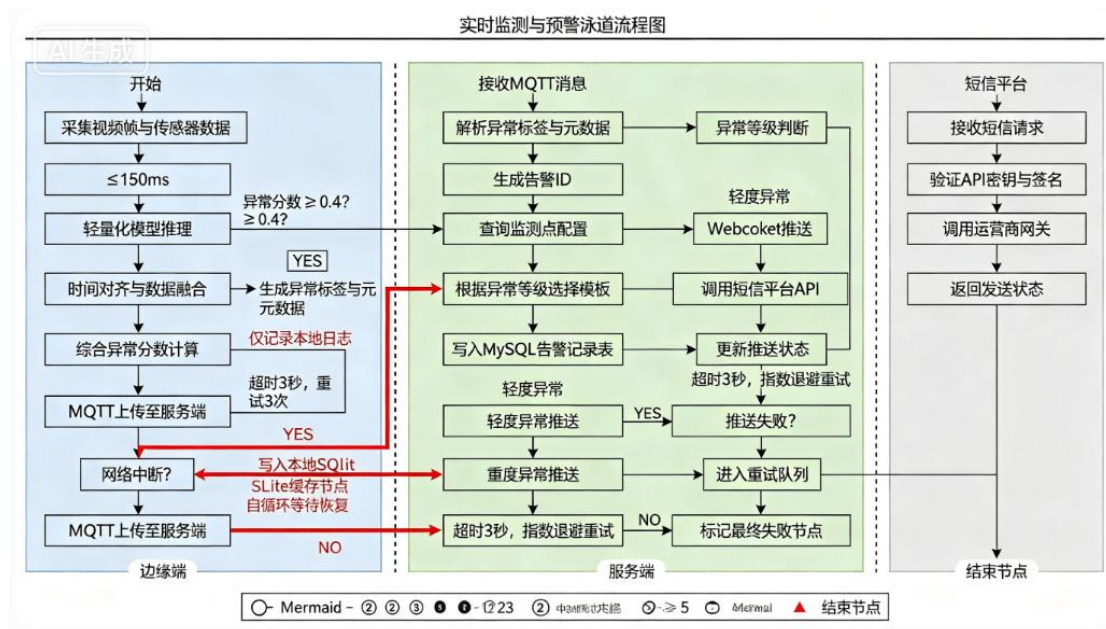
```

return {
    "final_score": final_score,
    "confidence": confidence,
    "alert_level": alert_level
}

```

当综合异常分数超过 0.4 时，系统进入预警触发子流程。首先为该异常生成唯一的告警 ID，然后查询监测点配置获取告警接收人列表。根据异常等级决定推送渠道，轻度异常仅写入告警记录表并推送到 Web 端平台消息，不触发短信。中度异常在轻度异常的基础上增加 WebSocket 推送，向当前在线用户实时发送通知。重度异常触发全部渠道，包括短信平台调用。

短信平台调用子流程采用异步方式，将发送请求放入线程池执行，避免阻塞主流程。发送成功后更新告警记录表中的 push_status 字段；发送失败时记录失败原因，并将告警 ID 放入重试队列。重试队列中的任务由独立的后台线程扫描，按指数退避策略重试，重试间隔分别为 1 分钟、2 分钟、4 分钟、8 分钟、16 分钟，5 次全部失败后标记为最终失败并发送运维告警。



5.2 数据同步与断点续传流程

边缘端与服务端之间的数据同步采用心跳保活与断点续传机制。边缘端启动后每 30 秒向服务端发送心跳包，心跳包包含设备 ID、时间戳与当前缓存队列长度。服务端收到心跳后回复心跳确认，若服务端超过 90 秒未收到心跳，则将该

监测点标记为离线。

网络正常时，边缘端每处理完一帧图像，立即将分析结果元数据通过 MQTT 上传至服务端。上传采用 QoS 等级 1，确保消息至少送达一次。服务端收到消息后返回确认，边缘端收到确认后继续处理下一帧。若上传超时或收到服务端的拒绝响应，边缘端将该条数据写入本地 SQLite 数据库作为缓存。

本地缓存采用 SQLite 数据库，表结构与云端 MySQL 对应，包含 water_quality_data 和 alert_record 两张表。缓存数据库设置最大容量为 500 MB，当容量接近上限时，系统按照先进先出原则删除最旧的记录，优先保留异常数据。网络恢复检测采用 TCP 连接测试机制，每隔 10 秒尝试向服务端发送探测包，连续三次成功则判定网络已恢复。

网络恢复后，边缘端启动缓存补传流程。补传流程从 SQLite 数据库中读取未上传的记录，按时间戳升序排序，逐条发送至服务端。每条记录发送成功后从本地缓存中删除，发送失败则保留并跳过该条记录，避免单条失败阻塞后续补传。补传过程中若网络再次中断，补传流程自动暂停，待下次网络恢复后从中断点继续。

5.3 用户认证与授权流程

用户认证流程采用 JWT 无状态认证机制。用户通过前端登录页面提交用户名与密码，前端对密码进行前端 MD5 哈希后发送至后端。后端接收到请求后，从数据库中查询用户信息，使用 BCrypt 算法验证密码哈希值。验证通过后，系统生成 JWT 令牌，令牌载荷中包含 userId、username、role 与 exp 过期时间。JWT 使用 HS256 算法签名，密钥存储在环境变量中。

令牌生成后，后端将其返回给前端。前端收到令牌后存储在 localStorage 中，并在后续所有请求的 HTTP 头中添加 Authorization 字段，值为 Bearer 加空格加令牌字符串。对于敏感操作，后端还会验证 CSRF 令牌，防范跨站请求伪造攻击。用户登出时，前端删除 localStorage 中的令牌，并向后端发送登出请求，后端将该令牌加入黑名单，黑名单中的令牌在剩余有效期内将被拒绝访问。

授权流程在每次 API 请求时执行。后端首先解析 JWT 令牌，提取用户 ID 与角色信息。然后根据请求的 API 路径与 HTTP 方法，查找该接口所需的权限配置。接口权限配置定义在注解中，例如@PreAuthorize(hasRole=admin)表示仅管理员

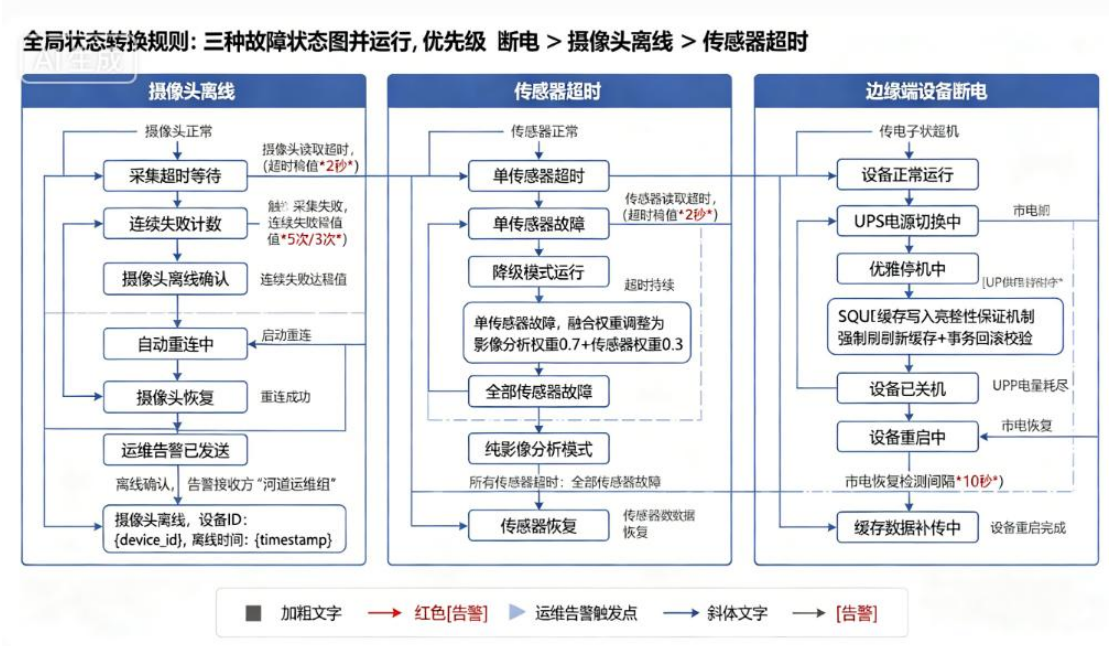
可访问。用户角色与接口要求匹配时放行请求，不匹配时返回 403 状态码。对于需要验证监测点所有权的接口，系统还需检查 user_point_permission 表中是否存在该用户与监测点的关联关系。

5.4 设备故障处理流程

设备故障包括摄像头离线、传感器超时与边缘端设备断电三类。摄像头离线检测通过视频流读取超时实现，连续 5 次读取失败后判定摄像头离线。此时系统记录故障日志，并通过 MQTT 向服务端上报设备故障事件。服务端收到事件后生成设备告警，发送给运维人员。

传感器超时检测通过 Modbus 读取超时实现。传感器数据采集子模块每次读取操作设置 2 秒超时，超时后重试一次，连续 3 次超时判定该传感器故障。单传感器故障不影响其他功能，系统继续使用剩余传感器数据结合影像分析进行水质判断。所有传感器均故障时，系统降级为纯影像分析模式。

边缘端设备断电前，系统通过 UPS 电源检测到电源切换事件，触发优雅停机流程。优雅停机流程首先停止新数据的采集，然后将内存中未处理的数据帧与缓存队列中的元数据强制写入本地 SQLite 数据库。写入完成后发送最后一条离线通知给服务端，告知预计离线时长。重新上电后，系统首先检查 SQLite 数据库中的未上传记录，优先补传断电期间缓存的数据，再恢复正常采集流程。



六、算法设计

6.1 轻量化模型推理算法

系统采用 MobileNetV2 与 U-Net 组合模型进行水质影像分析。MobileNetV2 的核心算子为深度可分离卷积，将标准卷积分解为深度卷积与逐点卷积两个步骤。深度卷积对每个输入通道单独进行卷积操作，逐点卷积使用 1×1 卷积核将深度卷积的输出进行通道组合。这种分解方式使得计算量从输入通道数乘以输出通道数乘以卷积核尺寸的平方，降低为输入通道数乘以卷积核尺寸的平方加上输入通道数乘以输出通道数，计算量减少约 8 到 9 倍。

模型量化方面，系统采用训练后动态量化技术。TensorFlow Lite 将模型权重从 32 位浮点数转换为 8 位整数，输入输出张量仍保持浮点格式。量化过程使用校准数据集统计各层激活值的动态范围，确定最优的缩放因子与零点。量化后的模型体积从原始 14.5 MB 压缩至 3.8 MB，推理速度提升约 40%，在树莓派 4B 上单帧推理时间从 210 毫秒降至 145 毫秒，满足 200 毫秒以内的性能要求。

U-Net 分割网络采用轻量化设计，编码器部分复用 MobileNetV2 的前若干层作为特征提取器，解码器部分采用转置卷积进行上采样。跳跃连接将编码器各层的特征图与解码器对应层拼接，保留空间细节信息。输出层使用 1×1 卷积与 softmax 激活函数，生成每个像素属于各类别的概率。分割结果经过后处理，包括形态学开运算去除孤立噪点，连通域分析提取污染区域边界框。

模型输入输出规格：

MobileNetV2 分类模型的输入张量形状固定为 $[1, 224, 224, 3]$ （批量大小 $\times$ 高度 $\times$ 宽度 $\times$ 通道数），

像素值在预处理阶段归一化至 $[-1, 1]$ 区间。模型输出为 $[1, 5]$ 的概率向量，五个输出节点依次对应：

索引 0-清晰、索引 1-轻度浑浊、索引 2-中度浑浊、索引 3-重度浑浊、索引 4-有机污染物检测。

推理时取 argmax 确定最终类别，同时输出该类别对应的置信度分数。

U-Net 分割模型的输入形状同样为 $[1, 224, 224, 3]$ ，输出形状为 $[1, 224, 224, 4]$ 。

四个输出通道的含义为：通道 0-背景概率、通道 1-油污区域概率、通道 2-泡沫/浮渣区域概率、通道 3-浑浊水体概率。

最终分割掩码通过对通道维度取 argmax 获得，每个像素被归类为概率最高

的类别。

损失函数与训练配置：

分类任务使用分类交叉熵损失，分割任务使用 Dice Loss 与交叉熵损失的加权和（权重比为 1:1）。

模型使用 AdamW 优化器，初始学习率为 $1e-4$ ，权重衰减为 $1e-5$ ，批量大小为 32。

训练过程中应用以下数据增强策略：随机旋转角度范围 $\pm 15^\circ$ ，随机水平翻转概率 0.5，

随机亮度调整范围 $\pm 20\%$ ，随机对比度调整范围 $\pm 20\%$ 。

评估指标：

分类任务评估指标包括准确率和宏平均 F1 分数。分割任务评估指标包括平均交并比（mIoU），

目标为在验证集上达到 $mIoU \geq 0.75$ 。模型量化后推理延迟在树莓派 4B 上不超过 200 毫秒，

在 Jetson Nano 上不超过 150 毫秒。

6.2 多源数据融合算法

多源数据融合采用加权投票算法，融合影像分析结果与传感器分析结果。影像分析输出浑浊度等级 t 以及各污染物类型的概率向量 p 。传感器分析输出浊度异常等级 s_{turb} 、COD 异常等级 s_{cod} 与 pH 异常等级 s_{ph} 。将各异常等级转换为数值分数，正常对应 0 分，轻度对应 0.33 分，中度对应 0.67 分，重度对应 1 分。传感器综合分数取三个传感器分数的加权平均，浊度权重设为 0.5，COD 设为 0.3，pH 设为 0.2，依据各指标对水质评价的重要性确定。

加权投票的权重设定为影像分析权重 0.6，传感器分析权重 0.4。融合后的综合异常分数计算公式为 $score = 0.6 * image_score + 0.4 * sensor_score$ 。影像分析分数 $image_score$ 由浑浊度等级分数乘以 0.7 加上最高污染物类型概率乘以 0.3 组成，既反映整体浑浊程度又兼顾特定污染物。融合结果的置信度根据影像分析与传感器分析的一致性程度计算，两者异常等级差距超过 2 级时置信度降低 30%。

时间对齐策略以影像帧的时间戳为基准，在指定时间窗口内搜索传感器读数。

时间窗口宽度设定为 500 毫秒，窗口内存在多个传感器读数时取平均值，窗口内没有传感器读数时使用最近一次有效读数，最近读数超过 10 秒时标记传感器数据缺失，融合时传感器权重降为 0。

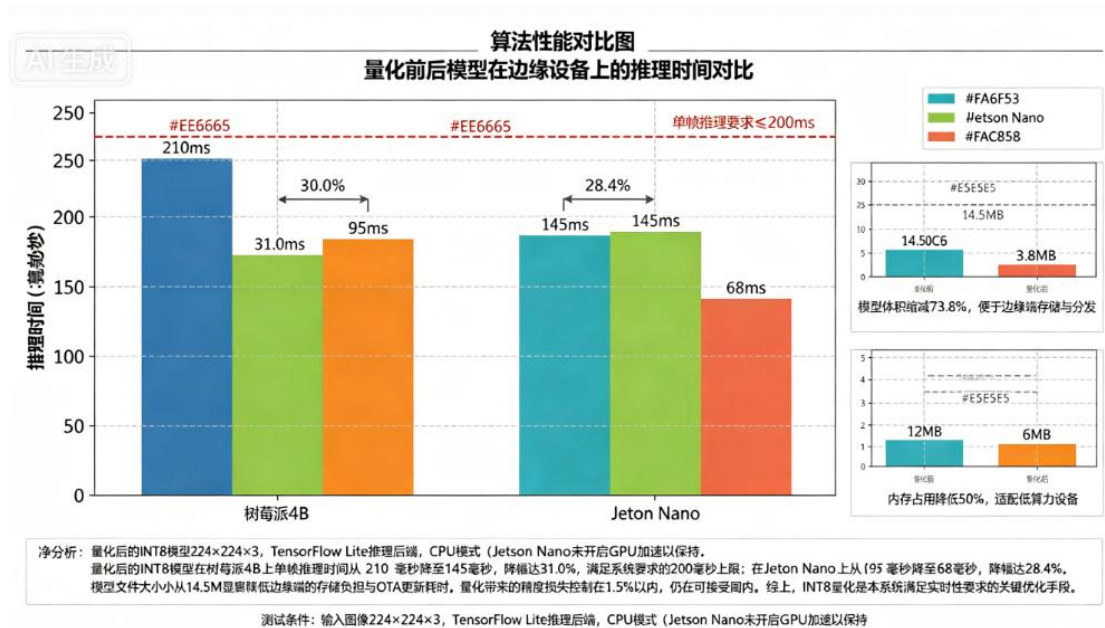
6.3 边缘端缓存淘汰算法

边缘端本地缓存采用 LRU 变种算法进行淘汰管理。缓存分为两个区域，热数据区存放最近访问的高频数据，冷数据区存放低频数据。新写入的数据首先进入冷数据区，被访问两次后晋升到热数据区。热数据区采用 LRU 淘汰，冷数据区采用 FIFO 淘汰。这种设计避免单次突发访问污染缓存，保留真正长期热点的数据。缓存容量限制为 500 MB，系统每 5 分钟检查一次当前缓存使用量。超过容量上限时触发淘汰流程，优先淘汰冷数据区中最早写入的记录，每次淘汰 100 条记录后重新检查容量。淘汰记录时同步更新索引文件，确保缓存文件与索引的一致性。对于包含异常标签的记录，系统设置保护机制，异常等级为中度或重度的记录在缓存中保留至少 7 天，不受容量淘汰限制，确保重要数据不丢失。

缓存数据的序列化采用 MessagePack 格式，相比 JSON 体积更小且支持二进制数据直接存储。每个缓存条目包含元数据与原始数据两部分，元数据存储于 SQLite 数据库中，原始图像文件存储在文件系统中，通过路径关联。这种分离存储策略便于快速检索元数据，同时避免大文件占用数据库空间。

6.4 算法复杂度分析

MobileNetV2 模型单次推理的时间复杂度为 $O(\text{输入尺寸} \times \text{通道数} \times \text{卷积核尺寸的平方} / \text{压缩因子})$ 。对于 $224 \times 224 \times 3$ 的输入图像，浮点运算次数约为 3 亿次，量化后降至约 1.8 亿次。空间复杂度方面，模型权重存储占用约 3.8 MB，中间激活张量占用约 2 MB，总计不超过 6 MB，能够在边缘端内存受限环境下运行。



多源数据融合算法的时间复杂度为 $O(n+m)$ ，其中 n 为时间窗口内的影像帧数量， m 为传感器读数数量。由于时间窗口固定为 1 秒， n 与 m 均为常数，因此融合算法的时间复杂度可视为 $O(1)$ 。空间复杂度同样为 $O(1)$ ，仅需存储常数个中间变量。

缓存淘汰算法的时间复杂度为 $O(\log N)$ ，其中 N 为缓存条目数量。LRU 变种算法使用双向链表与哈希表的组合结构，访问操作在链表中查找节点的时间复杂度为 $O(1)$ ，晋升操作需要更新链表指针同样为 $O(1)$ 。淘汰操作需要找到冷数据区尾部节点，由于冷数据区采用 FIFO 队列，获取尾部节点的时间复杂度为 $O(1)$ ，整体操作复杂度为常数级别。

七、异常处理设计

7.1 错误码体系设计

系统定义五层错误码结构，每层错误码由四位数字组成。第一位数字表示错误类型，1 表示客户端错误，2 表示服务端错误，3 表示设备端错误，4 表示第三方服务错误，9 表示系统级错误。第二位数字表示具体模块，0 表示通用模块，1 表示认证授权模块，2 表示数据采集模块，3 表示智能分析模块，4 表示预警推送模块，5 表示数据存储模块。后两位数字表示具体错误序号。

客户端错误码范围 1000 到 1999。1001 表示请求参数缺失或格式错误，1002 表示认证失败，1003 表示权限不足，1004 表示请求资源不存在，1005 表示请求方法不支持，1006 表示请求体过大，1007 表示请求频率超限。服务端错误码范

围 2000 到 2999。2001 表示模型推理失败，2002 表示数据库连接异常，2003 表示数据库查询超时，2004 表示事务提交失败。设备端错误码范围 3000 到 3999。3001 表示摄像头离线，3002 表示传感器超时，3003 表示存储空间不足，3004 表示网络连接断开。第三方服务错误码范围 4000 到 4999。4001 表示短信平台调用失败，4002 表示地图 API 调用超时，4003 表示第三方服务返回异常。系统级错误码范围 9000 到 9999。9001 表示系统资源不足，9002 表示内部服务调用超时，9003 表示配置加载失败。

错误处理状态机：

对于可重试的错误（如 2002 数据库连接异常、4001 短信平台调用失败），系统采用指数退避重试策略，状态流转如下：

初始错误 → 记录次数 → 第 1 次重试(延迟 1 秒) → 第 2 次重试(延迟 2 秒) → 第 3 次重试(延迟 4 秒) → 第 4 次重试(延迟 8 秒) → 第 5 次重试(延迟 16 秒) → 最终失败 → 熔断

熔断器状态流转：

关闭状态(Closed) → 错误次数达到阈值(5 次) → 开启状态(Open, 持续 10 分钟) →

半开状态(Half-Open, 允许 1 个探测请求) → 成功则关闭，失败则重新开启

对于不可重试的错误（如 1003 权限不足、1004 资源不存在），系统直接返回错误响应，

不进入重试队列，前端根据错误码展示对应提示。

7.2 错误处理策略

输入校验异常采用快速失败策略。数据采集模块接收到图像帧后，首先校验图像格式是否为 JPEG 或 PNG，校验失败时丢弃该帧并记录警告日志，不进入后续处理流程。传感器数据读取后校验数值范围，pH 值超出 0 到 14 范围时标记为无效，使用最近一次有效值代替，连续无效超过 10 次则触发传感器故障告警。API 接口层使用 Spring Validation 框架对请求参数进行校验，校验失败时立即返回 400 错误码与具体的校验失败信息。

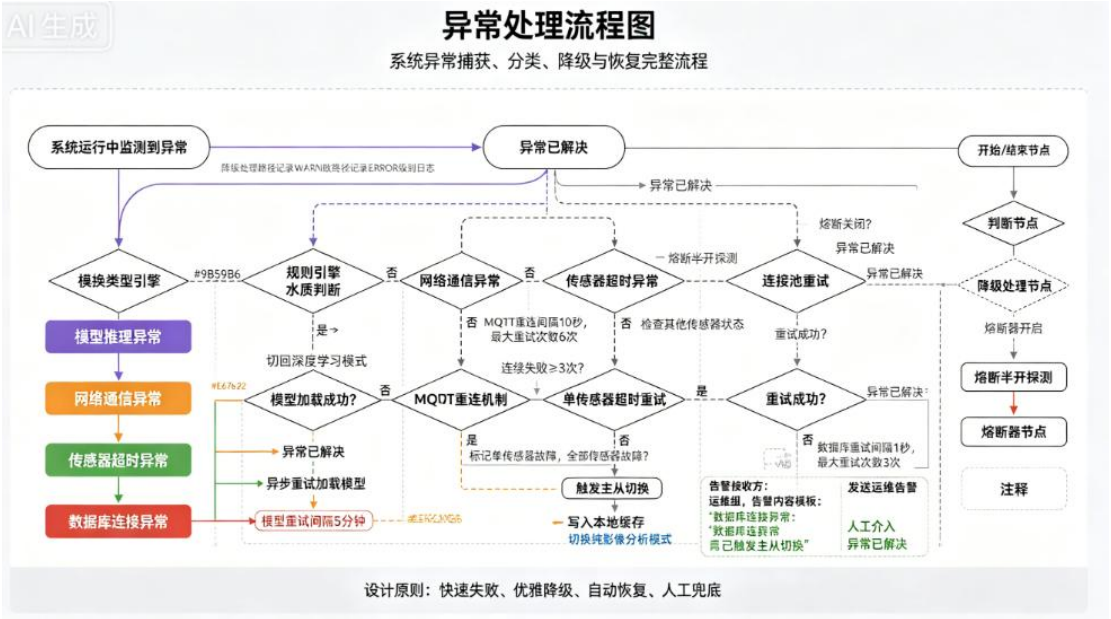
模型推理异常采用降级处理策略。TensorFlow Lite 推理过程中若抛出异常，系统捕获异常后记录错误堆栈，并自动切换到规则引擎模式。规则引擎基于传统

图像处理算法进行水质判断，使用颜色直方图分析水体浑浊度，虽然准确率低于深度学习模型，但能够保证系统在模型故障期间的基本运行。规则引擎模式下系统每 5 分钟尝试重新加载一次模型，加载成功后自动切回深度学习模式。

网络通信异常采用重试与熔断相结合的策略。MQTT 连接断开时，边缘端每 10 秒尝试重连一次，重连成功后自动订阅相关主题。服务端调用短信平台 API 时设置 3 次重试机制，重试间隔采用指数退避，若全部失败则熔断该短信渠道 10 分钟，熔断期间所有短信推送需求仅记录日志不实际发送。熔断器半开状态下允许单个请求通过以探测服务是否恢复。

7.3 日志管理设计

系统采用 Log4j2 作为日志框架，配置异步日志记录器提升性能。日志级别分为 TRACE、DEBUG、INFO、WARN、ERROR、FATAL 六个级别。生产环境日志级别设置为 INFO，DEBUG 级别仅在开发与测试环境开启。ERROR 级别日志记录所有异常堆栈信息，WARN 级别记录可恢复的异常情况，INFO 级别记录关键业务流程节点。



日志输出采用滚动文件策略，单个日志文件最大 100 MB，达到上限后自动滚动生成新文件，保留最近 30 个日志文件。日志文件按天分目录存储，目录格式为 logs/yyyy/MM/dd。ERROR 级别日志单独写入 error.log 文件，便于快速定位严重问题。所有日志文件使用 GZIP 压缩，降低存储空间占用。

日志内容包含统一格式的时间戳、日志级别、线程名称、类名、方法名、行

号与消息内容。敏感信息如用户密码、JWT 令牌等在日志中自动脱敏，使用星号替换关键字符。系统还支持结构化日志输出，将日志以 JSON 格式写入，便于接入 ELK 等日志分析平台进行集中检索与监控。

八、安全设计

8.1 用户认证与权限管理

系统采用 JWT 令牌实现无状态认证，令牌有效期为 24 小时。令牌生成时使用 HMAC-SHA256 算法进行签名，签名密钥长度不少于 256 位，存储在环境变量中不写入代码仓库。令牌载荷中包含用户 ID、用户名、角色与权限列表，为减小令牌体积，权限列表仅存储权限编码的整数形式。令牌刷新机制采用滑动过期策略，用户在令牌过期前 30 分钟内发起请求自动刷新令牌，返回新令牌供后续请求使用。

权限管理采用基于角色的访问控制模型，细粒度到每个 API 接口。管理员角色拥有所有接口的访问权限，包括用户管理、监测点配置与系统参数设置。普通用户角色仅能访问其所属监测点的数据查看与告警确认接口。只读用户角色仅能访问数据查询接口，无法进行任何修改操作。权限校验通过 AOP 切面实现，在控制器方法执行前解析 JWT 并比对权限。

8.2 数据加密与保护

用户密码采用 BCrypt 算法进行哈希存储，每个密码使用独立的随机盐值，计算成本因子设置为 10。BCrypt 算法输出长度为 60 字符，包含盐值与哈希值，破解难度远高于简单 MD5。用户登录时对输入的密码进行同样的哈希计算，与存储值比较，哈希计算过程在服务端执行，前端仅做简单的 MD5 预处理，不减轻密码传输安全性。

敏感配置信息如数据库密码、API 密钥等使用 AES-256 算法加密存储。加密密钥存储在专用的密钥管理服务中，应用程序启动时从密钥管理服务获取解密密钥，密钥在内存中以字节数组形式存在，使用后立即擦除。配置文件中的敏感字段存储为密文，应用启动时自动解密加载。

传感器数据传输采用 MQTT over TLS，TLS 版本不低于 1.2，使用服务器证书验证机制，禁止降级到不安全协议版本。边缘端与服务端之间的所有网络通信均经过加密，包括心跳包与遥测数据。前端与后端之间的 HTTPS 通信配置 HSTS

策略，强制浏览器使用 HTTPS 访问，防范中间人攻击。

8.3 安全漏洞防护

SQL 注入防护通过 MyBatis 的参数化查询实现。所有数据库查询均使用预编译语句或 `{ param }` 占位符，避免使用 `${ param }` 拼接用户输入。对于需要动态表名与字段名的场景，系统使用白名单校验，仅在预定义的可选值范围内进行拼接。MyBatis Plus 的查询包装器同样使用参数化查询，自动转义特殊字符。

跨站脚本攻击防护通过输出编码实现。前端 Vue 框架默认对插值表达式中的内容进行 HTML 转义，避免用户输入被解析为脚本。对于需要动态渲染 HTML 的场景，系统使用 DOMPurify 库对内容进行净化。后端在返回用户输入内容时同样进行转义处理，双重保障防范 XSS 攻击。

跨站请求伪造防护采用同步令牌模式。服务端在用户登录成功后生成 CSRF 令牌，存储在 Cookie 中并同页面返回。前端发起非 GET 请求时从 Cookie 中读取 CSRF 令牌，添加到请求头的 X-CSRF-TOKEN 字段中。服务端校验请求头中的令牌与 Cookie 中的令牌是否一致，不一致则拒绝请求。AJAX 请求自动携带同源 Cookie，确保令牌能够正确传递。

九、可扩展性设计

9.1 模块化与可插拔设计

系统采用微服务架构，各模块之间通过定义良好的接口进行通信，实现低耦合。数据采集模块可替换为不同厂商的摄像头驱动，只需实现统一的 CameraAdapter 接口，该接口定义 startCapture、stopCapture 与 getFrame 三个方法。系统通过配置文件指定使用的适配器类名，运行时使用 Java 反射机制动态加载。新增传感器类型同样通过扩展 SensorAdapter 接口实现，无需修改核心业务逻辑。

预警推送模块支持推送渠道的插件化扩展。系统定义 PushChannel 接口，包含 send 与 getStatus 两个方法。短信推送、WebSocket 推送与平台消息推送均实现该接口。新增邮件推送或钉钉机器人推送时，只需实现 PushChannel 接口并在配置文件中注册即可。接口管理器在启动时扫描所有实现了 PushChannel 接口的类，自动加载到推送渠道列表中。

模型推理模块支持多种推理后端的热插拔。系统定义 InferenceEngine 接口，

包含 loadModel、infer 与 unloadModel 三个方法。TensorFlow Lite、ONNX Runtime 与 OpenVINO 分别实现该接口。系统根据边缘端硬件类型自动选择最优后端，Jetson Nano 上使用 TensorRT 后端可获得最佳性能，树莓派上使用 TensorFlow Lite 后端兼顾性能与兼容性。

9.2 平台兼容性设计

系统边缘端兼容树莓派 4B 与 Jetson Nano 两种主流开发板。针对不同平台，系统提供独立的 Docker 镜像，镜像内预编译对应的推理库与驱动程序。树莓派镜像基于 Raspberry Pi OS ARM64 构建，安装 ARM 架构的 TensorFlow Lite 库。Jetson Nano 镜像基于 JetPack SDK 构建，利用 CUDA 加速库提升推理性能。系统启动时读取设备树信息识别硬件平台，自动加载对应的模型量化参数。

前端应用采用响应式设计，兼容 Web 端与移动端浏览器。使用 Element Plus 组件库提供的响应式栅格系统，根据不同屏幕宽度自动调整布局。移动端触摸事件经过优化，按钮尺寸不小于 44×44 像素，符合移动端交互规范。移动端还支持离线模式，使用 Service Worker 缓存静态资源，网络不佳时仍可查看已加载的页面与缓存的数据。

后端服务全平台兼容，支持在 Linux 与 Windows Server 上部署。所有路径使用 File.separator 处理，避免硬编码反斜杠或正斜杠。环境变量与配置文件分离，不同环境通过 Spring Profile 切换配置。数据库脚本兼容 MySQL 5.7 及以上版本，不使用特定版本的语法特性。

9.3 扩展点设计

系统预留监测点数量扩展能力。数据采集模块采用异步非阻塞模型，单个边缘端设备理论支持同时接入 4 路摄像头与 16 路传感器。当监测点数量需要扩展时，可通过增加边缘端设备数量水平扩展，云端服务支持自动发现新接入的监测点。服务端通过乐观锁与分布式锁协同多设备写入，避免数据冲突。

系统预留模型更新扩展点。智能分析模块支持在线模型更新，边缘端定期向服务端查询模型版本号，发现新版本时自动下载并热加载。模型更新采用双缓冲机制，新模型在后台加载完成后原子切换，不影响正在进行的推理任务。下载失败或加载失败时回退到原有模型，保证系统持续可用。

系统预留第三方数据源接入扩展点。多源数据融合模块设计为可配置的数据

10.2 数据库优化策略

数据库写入性能优化采用批量提交与异步写入相结合的策略。边缘端将多条水质数据积攒到一定数量后批量上传，批量大小设置为 50 条，减少网络往返次数。服务端接收到批量数据后使用 JDBC 批处理写入，单次提交包含 500 条记录，显著提升写入吞吐量。写入操作使用异步非阻塞方式，将数据放入队列后立即返回，后台线程池负责实际写入，避免阻塞主业务线程。

索引优化方面，定期使用 EXPLAIN 分析慢查询，针对性地增加覆盖索引。水质数据表的 (point_id, timestamp) 复合索引覆盖了 95% 的查询场景，查询时能够直接从索引返回结果，避免回表操作。生产环境开启慢查询日志，超过 500 毫秒的查询记录到单独文件，定期评审并优化。对不再使用的索引及时删除，减少写入时的索引维护开销。

分页查询优化采用延迟关联技术。先查询符合条件的记录主键 ID，再通过主键 ID 关联查询完整记录。这种方法避免了大偏移量时 MySQL 需要扫描大量数据的问题。对于深度分页场景，系统限制最大页码为 100 页，超过后提示用户缩小查询范围。导出功能使用流式查询，每次从数据库中读取 1000 条记录，处理完成后再读取下一批，避免全部加载到内存中。

10.3 异步处理与并发优化

边缘端采用多线程并发处理图像帧。线程池大小为 CPU 核心数的两倍，对于树莓派 4B 四核处理器，线程池大小设置为 8。图像采集线程将帧放入阻塞队列，工作线程从队列中取出进行推理。队列大小设置为 30，队列满时采集线程短暂阻塞，实现背压控制。使用 CompletionService 管理推理任务的完成顺序，先完成的任务先处理，提升响应速度。

服务端接口采用异步 Servlet 处理长时间操作。预警推送模块的短信发送使用 @Async 注解标记为异步方法，配置独立的线程池，核心线程数 10，最大线程数 50，队列容量 200。发送完成后通过回调函数更新数据库状态。前端轮询查询发送结果，超时时间设置为 10 秒，避免用户无限等待。

模型推理并发采用模型副本策略。多核设备上加载多个模型副本，每个推理线程绑定一个专属副本，避免多线程同时访问模型解释器造成的线程阻塞。模型副本数量与工作线程数量相同，每个副本独立的内存空间，互不干扰。模型加载

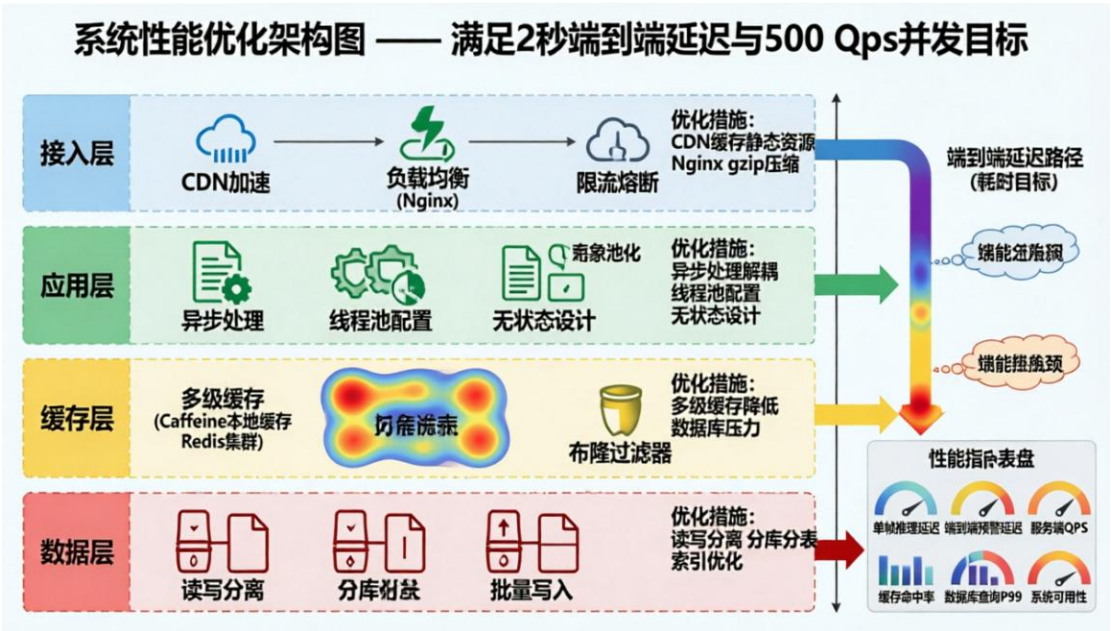
过程在启动时完成，运行时不再动态创建副本，保证推理过程的确定性。

10.4 负载均衡与集群扩展

服务端采用 Nginx 作为反向代理与负载均衡器。Nginx 配置 upstream 模块，使用加权轮询算法分发请求到多个后端实例。权重根据后端实例的 CPU 核心数与内存大小动态配置，高性能实例分配更高的权重。Nginx 开启 gzip 压缩，对 JSON 响应进行压缩，压缩级别设置为 5，在压缩率与 CPU 消耗之间取得平衡。

会话管理采用无状态设计，JWT 令牌携带所有必要信息，服务端不存储会话状态。这种设计使得服务端实例可以任意扩缩容，无需考虑会话同步问题。对于某些需要临时状态的场景，使用 Redis 集中存储，所有实例共享同一份状态数据。无状态实例通过 Kubernetes 的 Horizontal Pod Autoscaler 自动伸缩，根据 CPU 使用率与请求 QPS 动态调整实例数量。

数据库读写分离与分库分表。写操作统一路由到主库，读操作路由到从库集群。从库数量可以动态增减，使用负载均衡算法分发读请求。water_quality_data 表数据量达到千万级别时，系统自动触发分表操作，按月分表，每月数据独立存储。查询时根据时间范围确定需要查询的表，使用数据库中间件屏蔽分表逻辑。



十一、测试方案设计

11.1 单元测试设计

单元测试采用 JUnit 5 框架，覆盖核心业务逻辑与边界条件。智能分析模块的融合算法编写独立测试用例，覆盖正常融合、传感器数据缺失、传感器数据过

期等场景。每种场景设计至少三个测试数据，包括正常值、边界值与异常值。浮点数比较使用 Delta 误差范围，避免精度问题导致测试失败。测试用例命名遵循 `methodName_scenario_expectedBehavior` 约定，使测试意图清晰明确。

权限管理模块的 JWT 认证编写参数化测试。使用 `CsvSource` 注解提供多组测试参数，包括有效令牌、过期令牌、签名错误令牌、空令牌等场景。每个测试用例独立验证后端的响应状态码与错误码是否符合预期。对于敏感操作如删除用户，额外测试权限不足与越权访问场景，确保安全机制正确生效。

模型推理单元测试使用模拟输入输出。不实际加载完整模型，而是使用 Mock 对象模拟模型推理结果，聚焦于推理结果的后续处理逻辑是否正确。对于模型加载失败场景，测试降级到规则引擎的逻辑是否正确，包括模型加载重试次数、回退机制触发条件与恢复逻辑。

11.2 集成测试设计

模块间集成测试采用 Spring Boot Test 框架，启动完整的应用上下文并模拟外部依赖。测试数据采集模块与智能分析模块的集成时，使用嵌入式摄像头模拟器向数据采集模块推送测试图像，验证智能分析模块是否正确接收并返回分析结果。嵌入式摄像头模拟器能够模拟多种光照条件与污染场景的图像，覆盖概要设计中特殊输入分析的所有情况。

云端与边缘端的集成测试使用测试 MQTT Broker。边缘端实例在测试环境中连接到测试 Broker，发送模拟的水质数据。云端订阅相应主题，验证数据接收、存储与预警推送的完整流程。测试覆盖网络中断与恢复场景，验证断点续传机制是否正确，包括缓存数据的完整性、补传顺序与重复数据处理。

数据库集成测试使用 H2 内存数据库，测试完成后自动回滚事务，不影响开发环境数据。测试覆盖所有数据库操作，包括 CRUD 操作、事务边界与并发写入场景。并发测试使用 `CountDownLatch` 与 `ExecutorService` 模拟多线程同时写入，验证数据库锁机制与隔离级别是否满足要求。

11.3 性能与压力测试设计

11.3.1 测试工具与环境

测试工具：

- JMeter 5.6+：模拟并发请求，生成压力负载

- Prometheus + Grafana：监控 CPU、内存、网络 IO、数据库连接数
- tc (traffic control)：模拟网络丢包和延迟

测试环境配置：

- 云端服务：2 个 4 核 8G 实例，部署后端服务
- 数据库：8 核 16G MySQL 8.0，主从架构
- 边缘端：树莓派 4B 4GB + Jetson Nano 4GB 各 1 台
- 网络：内网千兆带宽

11.3.2 测试场景与通过标准

场景一：单监测点长稳测试

测试方法：模拟 1 个监测点，以每秒 1 帧的速度持续上报数据，运行 24 小时。

通过标准：

指标	阈值	监控方式
边缘端内存占用	$\leq 60\%$	Prometheus
端到端延迟 P99	≤ 2 秒	JMeter 聚合报告
内存泄漏	24 小时内内存增长 $\leq 5\%$	每小时采样记录
系统崩溃 / 重启	0 次	日志实时监控

场景二：并发接入压力测试

测试方法：逐步增加模拟监测点数量（10 → 50 → 100 → 200 → 500），每个点每秒上报 1 条数据。

通过标准：

指标	阈值
最大并发监测点数	≥ 200 个
API 吞吐量 (/api/data 上报接口)	≥ 1000 TPS
错误率 (HTTP 5xx)	$\leq 0.1\%$
服务端 CPU 使用率峰值	$\leq 80\%$
服务端内存使用率峰值	$\leq 85\%$
数据库连接数	$\leq$ 连接池最大值的 90%

场景三：极端场景测试

测试方法：

1. 突发流量：100 个监测点在 10 秒内同时上线并上报历史缓存数据（每个点 50 条）。

通过标准：MQTT Broker 连接数不超限，无连接拒绝；数据库连接池不耗尽。

2. 网络波动：使用 tc 命令模拟 30%丢包率 + 200ms 延迟，持续运行 30 分钟。

通过标准：数据最终一致，无数据丢失；断点续传机制触发；网络恢复后 5 分钟内补传完成。

3. 数据库故障转移：模拟主库宕机。

通过标准：系统自动切换至从库，图表查询功能正常；数据写入返回友好错误提示，不崩溃。

11.3.3 压力测试执行命令示例

JMeter 命令行执行示例（无 GUI 模式）

```
jmeter -n -t water_quality_test.jmx -l test_result.jtl -Jthreads=100  
-Jrampup=60
```

网络模拟命令（在边缘端执行）

```
tc qdisc add dev eth0 root netem loss 30% delay 200ms
```

取消网络模拟

```
tc qdisc del dev eth0 root
```

11.3.4 测试报告要求

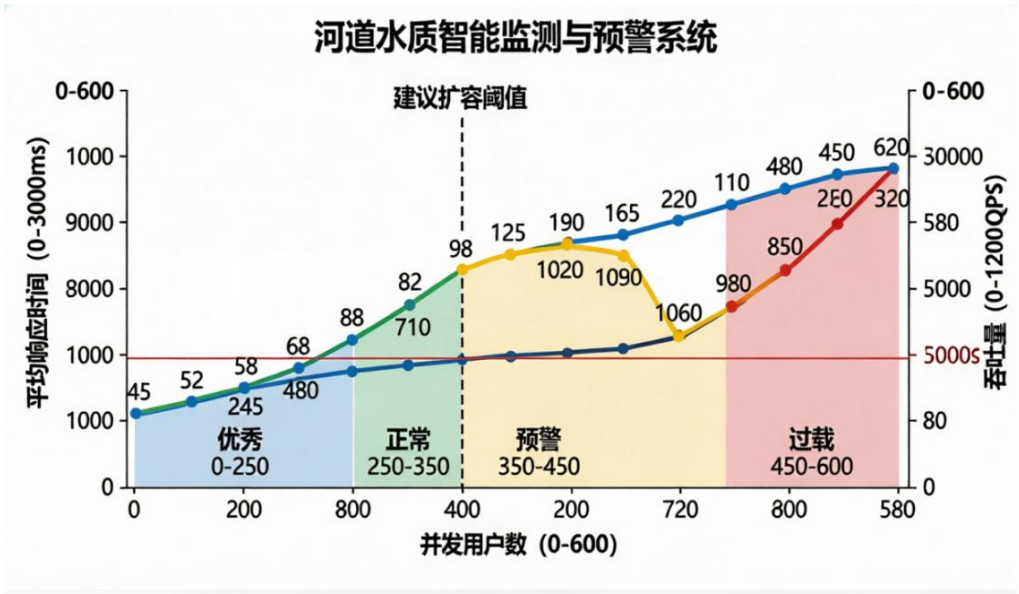
每次压力测试后需生成包含以下内容的报告：

1. 测试配置（并发数、持续时间、场景描述）
2. TPS 与响应时间分布图（P50、P90、P99）
3. 服务器资源使用率曲线图（CPU、内存、磁盘 IO、网络 IO）
4. 数据库慢查询日志分析
5. 系统瓶颈分析结论
6. 优化建议

11.4 压力测试通过标准

系统压力测试需满足以下全部指标方可判定通过。单帧模型推理延迟 99 分位值不超过 200 毫秒，在树莓派上不超过 200 毫秒，在 Jetson Nano 上不超过

150 毫秒。端到端预警推送延迟 99 分位值不超过 2 秒，包含图像采集、模型推理、数据上传、存储与推送的全部时间。系统支持不少于 100 个监测点同时在线，每个监测点每秒上报一组数据，服务端 CPU 使用率不超过 70%，内存使用率不超过 80%。



服务端并发处理能力不低于 500 QPS，接口正确率达到 99.9%以上，错误响应不高于 0.1%。压力测试过程中无系统崩溃或服务不可用事件发生，所有实例均能正常响应请求。资源释放正确，无内存泄漏或文件句柄泄漏现象。测试完成后系统能够恢复正常运行，无需人工干预重启。